SI 201: Project 2 - Airbnb “Scraping”

Introduction

Starting in the 1990s, the San Francisco Bay Area has had an affordable housing shortage. The introduction of Airbnb caused serious concerns by taking potential rental units off the housing market. As of 2015, lawmakers have attempted to regulate this by requiring potential listers to receive a business license from the local government before offering short-term rentals. This allows the government to manage the number of short-term rental units in the city. However, the government will only investigate Airbnb and property listings after receiving a complaint.

In this project, you will simulate the work of an independent investigator: extracting public information, analyzing patterns, and validating compliance. You will extract structured data from Airbnb web pages using the BeautifulSoup Python library. Web pages are written in HTML, which is designed to display information on a browser rather than support data analysis. Your task is to use web scraping - the process of extracting unstructured HTML and converting it into organized data - to analyze Airbnb listings.

In practice, web-scraping often involves downloading webpages automatically. However, many websites, including Airbnb, forbid the use of web-scraping in their terms of service (see Appendix). **To prevent a violation of Airbnb’s terms of service, we’ve provided legally-obtained, static HTML files for you to use.** Please use the static files in the `html_files` folder, **DO NOT** send requests to the actual website.

Collaboration and Academic Integrity

You may work **individually** or in a **group of up to 3 students**.

- All members of your group must submit the same GitHub URL of the group’s repository on Canvas.
- **If you choose to collaborate**, please refer to the “Collaborating with GitHub” document that can be found in Canvas under `Files` > `Useful Docs` > `Collab_with_Github.pdf` **before starting.**
- Similar to homework, you may use GenAI. However, you must report that you used it and how you used it. We recommend using it to help you learn, not just do your work for you. You are responsible for all GenAI code that you use in your project.

Deliverables

Submit a link to your group’s Project 2 Github repository, which must include:

- All committed code for Project 2.
- The output `.csv` file.
- One reflection file from each team member (`reflective_<first_name>.txt` or `reflective_<first_name>.pdf`).

Tasks

Functions

Implement all the functions below. We recommend reading the Debugging Tips and Tricks section in the appendix before you start. These tips will be very helpful!

`load_listing_results(html_path) -> list[tuple]`

- This function takes in the argument `html_path` representing the path of the `search_results.html` file.
- This function should return a list of tuples in the format: `(listing_title, listing_id)`
  - `listing_id` corresponds to the numeric ID in the URL (i.e. https://www.airbnb.com/rooms/1944564 has the listing id "1944564")
- Example output:

```
[
    ('Loft in Mission District', '1944564'),
    ('Home in Mission District', '49043049'),
    …
]
```

`get_listing_details(listing_id) -> dict`

- This function takes in a `string` containing the `listing_id` of an Airbnb listing.
- This function should return a **nested dictionary**:
  - The **outer dictionary** should have one key: the `listing_id (string)`.
  - The value for that key should be an **inner dictionary** with the following keys:
    - `policy_number (str)` - This field should be one of the following: a policy/license number string, "Pending", or "Exempt".
      NOTE: This field can be found in the section about the host. This is a text field entered by the lister, so the raw value may vary. You should inspect the raw data and categorize it into the three categories above when appropriate.
    - `host_type (str)` - If "Superhost" appears, set this field to "Superhost" Otherwise, set this field to "regular".
    - `host_name (str)` - The host name(s).
      NOTE: Some listings may have multiple hosts. Capture both names when applicable (for example: "Seth And Alexa").
    - `room_type (str)` - This field must be one of: "Private Room", "Shared Room", or "Entire Room".
      NOTE: This data is not explicitly given on this page. Use the following criteria to categorize the data into these three fields:

        - "Private Room": the listing subtitle contains the word "Private".
        - "Shared Room": the listing subtitle contains the word "Shared".
        - "Entire Room": the listing subtitle contains neither "Private" nor "Shared".
    - `location_rating (float)` - This field stores the listing's Location rating from the ratings section of the listing page.
      NOTE: Some listings may not have any ratings. If you cannot find a location rating for a listing, return 0.0
- Example output:

```
{
    "1944564": {
        "policy_number": "2022-004088STR",
        "host_type": "Superhost",
        "host_name": "Brian",
        "room_type": "Entire Room",
        "location_rating": 4.9
    }
    …
}
```

`create_listing_database(html_path) -> list[tuple]`

- This function takes in the argument `html_path` representing the path of the `search_results.html` file.
- This function should return a list of tuples, where each tuple contains the complete information for one listing. Use `load_listing_results()` to get the `listing_title` and `listing_id`, then use `get_listing_details()` to get the remaining fields from the nested dictionary, and combine them into a tuple in this order:

```
(listing_title, listing_id, policy_number, host_type, host_name, room_type, location_rating)
```

- Example output:

```
[
    ('Loft in Mission District', '1944564', '2022-004088STR', 'Superhost', 'Brian', 'Entire Room', 4.9),
    ('Home in Mission District', '49043049', 'Pending', 'Superhost', 'Cherry', 'Entire Room', 4.9),
    …
]
```

`output_csv(data, filename)-> None`

- Write a function that takes in a list of tuples called `data` (i.e., the one returned by `create_listing_database()`) and a string `filename` representing the name of the output csv file.
- This function should sort the tuples in descending order by `location_rating`, then write the sorted data to a CSV file and save it to the passed filename. The first row of the csv should contain "Listing Title", "Listing ID", "Policy Number", "Host Type", "Host Name", "Room Type", "Location Rating", respectively as column headers. For each tuple

in the data, write a new row to the csv, placing each element of the tuple in the correct column. The data should be written in the csv in descending order from the highest `location_rating` to the lowest `location_rating`.

- Expected structure in csv file:

```
Listing Title,Listing ID,Policy Number,Host Type,Host Name,Room Type,Location Rating
Loft in Mission District,1944564,2022-004088STR,Superhost,Brian,Entire Room,4.9
…
```

`avg_location_rating_by_room_type(data) -> dict`

- Write a function that takes in a list of tuples called `data` (i.e., the one returned by `create_listing_database()`) and calculates the average `location_rating` for each `room_type`.

  NOTE: Some listings may not have ratings and will have `location_rating` equal to 0.0. Exclude those rows before calculating the average.
- Expected output:

```
{
    'Private Room': 4.9,
    …
}
```

`validate_policy_numbers(data) -> list[str]`

- Write a function that takes in a list of Airbnb listing tuples called `data`, (i.e. the return value from `create_listing_database()`), and parses through the `policy_number` of each, validating if it matches the policy number format. Ignore any pending or exempt listings.
- Return a list of the `listing_id` if the corresponding policy numbers that do not match the correct format.
  - *Policy numbers* are a reference to the business license San Francisco requires to operate a short-term rental. These come in two forms below. # means any digit 0-9.
    - 20##-00####STR
    - STR-000####

Test cases

After you have implemented all required functions, you will need to write the corresponding test cases for each function.

`test_load_listing_results()`

- Check that the number of listings extracted is 18.
- Check that the first tuple is `("Loft in Mission District", "1944564")`.

`test_get_listing_details()`

- Call `get_listing_details()` on each provided listing id and save results in a list.
- Spot-check values by opening the corresponding `listing_<id>.html` files:
  - Check that the listing 467507 has the correct policy number "STR-0005349".
  - Check that the listing 1944564 has the correct host type "Superhost" and room type "Entire Room".
  - Check that listing 1944564 has the correct location rating 4.9.

`test_create_listing_database()`

- Check that each tuple in detailed_data has exactly 7 elements.
- Spot-check the last tuple is `("Guest suite in Mission District", "467507", "STR-0005349", "Superhost", "Jennifer", "Entire Room", 4.8)`.

`test_output_csv()`

- Call output_csv() to write the detailed_data to a CSV file.
- Read the CSV back in and store rows in a list.
- Check that the first data row matches `["Guesthouse in San Francisco", "49591060", "STR-0000253", "Superhost", "Ingrid", "Entire Room", "5.0"]`.

`test_avg_location_rating_by_room_type()`

- Call `avg_location_rating_by_room_type()` and save the output.
- Check that the average for "Private Room" is 4.9.

`test_validate_policy_numbers()`

- Call `validate_policy_numbers()` on detailed_data and save the result.
- Check that the list contains exactly "16204265" for this dataset.

Reflective Questions

For this section, **each team member must submit their own reflection as a separate file.** You may write your responses in a `.txt` file or in another document format and export it to a `.pdf` file. Name your file `reflective_<first_name>.txt` or `reflective_<first_name>.pdf`. These questions are intentionally open-ended so there are no right or wrong answers. As long as your responses are paragraph-length, thoughtful, and well reasoned, you will receive full credit.

In this project, you used web-scraping to collect information from Airbnb to determine whether hosts complied with San Francisco's rental laws. These regulations rely on multiple parties to maintain accountability: hosts must register with the city's planning office, Airbnb must accurately display policy numbers, and third parties scrape data and file complaints.

The questions below ask you to reflect on whether the current system effectively holds hosts and platforms accountable, along with the legal and ethical issues associated with web-scraping.

- Do you think this current system provides adequate accountability around short-term rentals in San Francisco? In your response, describe why you believe the system is adequate or inadequate. If you believe the system is inadequate, discuss a possible change in actions required to improve accountability in this system. You may refer to the SWE Code of Ethics as you think through your response.

- As discussed in the introduction, the legality of web scraping is still uncertain in the US. Skim through the Legal Issues section of Web Scraping in the US on Wikipedia and this article about the legal issues with the Computer Fraud and Abuse Act, and describe at least one factor you believe is important to consider when discussing the legality of web scraping and why.

- Scraping public data does not always lead to positive results for society. For example, look to Facebook–Cambridge Analytica data scandal - Wikipedia or Clearview AI - Wikipedia. Many argue that using someone's personal data without their consent (even if publicly provided) is unethical. Web scraping requires thoughtful intervention. Develop at least two guidelines you think are important to consider when deciding whether to do web scraping.

Bonus Function

`google_scholar_searcher(query) -> list`

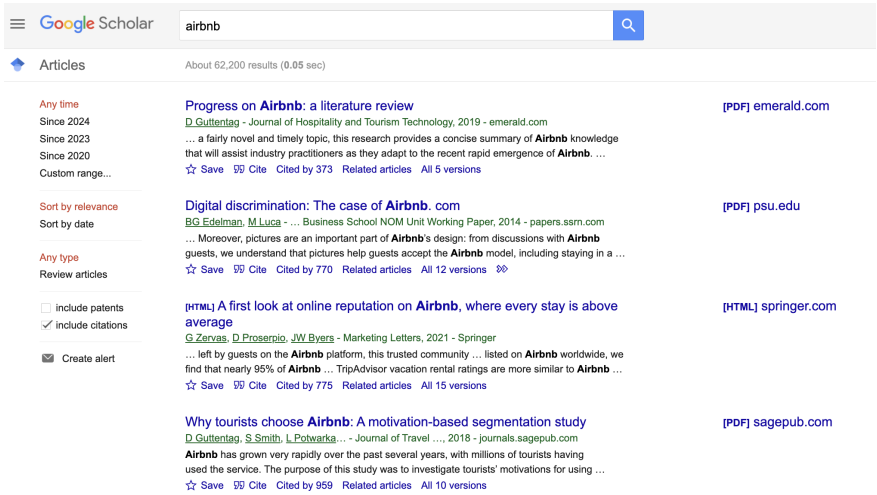
Although we worked with static HTML files due to legalities and Airbnb's terms of service, typically when working with web scraping, you make requests to a web server to retrieve data. For this extra credit question, you will use a Python module, `requests`, to send a request to the HTTP server and retrieve data from the server.

Parameter: query (`str`)
Return: a list of titles on the first page (`list`)

Write a function that imports the `requests` library of Python and sends a request to Google Scholar with the passed query. Using BeautifulSoup, find all titles and return the list of titles you see on page 1 (you do not need to scrape results on other pages).

NOTE: You do not need to write test cases for the extra point question. Your searching results might not be the same as the example research results shown here.

For example, for a search query "airbnb"



the function returns:

```
['Progress on Airbnb: a literature review',
 'Digital discrimination: The case of Airbnb. com',
 'A first look at online reputation on Airbnb, where every stay is above
average',
 'Why tourists choose Airbnb: A motivation-based segmentation study',
 'Who benefits from the" sharing" economy of Airbnb?',
 'Airbnb: the future of networked hospitality businesses',
 'The seven lives of Airbnb. The role of accommodation types',
 'Current state and development of Airbnb accommodation offer in 167
countries',
 'Airbnb: disruptive innovation and the rise of an informal tourism
accommodation sector',
 'A socio-economic analysis of Airbnb in New York City']
```

Grading

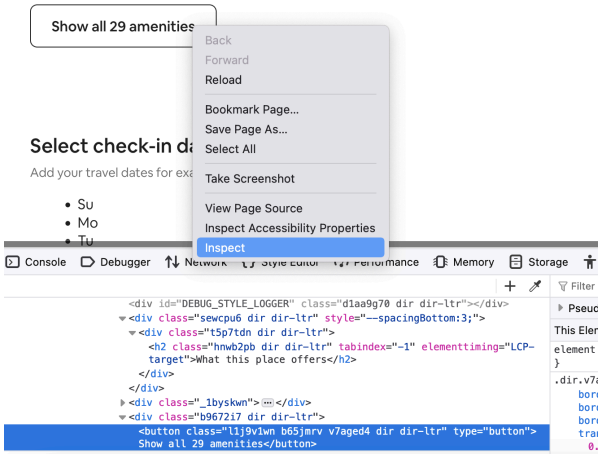
Item	points
Functions	
`def load_listing_results()`	20
`def get_listing_details()`	40
`def create_listing_database()`	15
`def output_csv(data, filename)`	10
`def avg_location_rating_by_room_type()`	10
`def validate_policy_numbers()`	15
Test Cases	
Test cases (10 points for each)	60
Reflective Thoughts	
Reflection that shows critical thought (10 points for each) (Graded independently)	30
Bonus Function	
`def google_scholar_searcher(query)`	20 (bonus)
Total	**200 + 20 (bonus)**

We will be checking to make sure that you've implemented each function correctly. You will need to make sure that you are returning data in the specified format to get full credit. You will also need to make sure that you are calling the other functions when directed to do so.

Appendix

Debugging Tips and Tricks

While you can open HTML files in VSCode (see StackOverflow for how to format code in VSCode), it is often easier to inspect the files **in your web browser** inspector tools (see the screenshot below). In Chrome and Firefox, you can access the inspector tools by right-clicking, then selecting "Inspect." In Safari, you must enable features for web developers (see here for more information).



Based on the example above, we can use our inspector tools to find out that if we're looking for the number of amenities, we might search for tags with the class "b9672i7 dir dir-ltr" or <button> tags with the class "l1j9v1wn b65jmrv v7aged4 dir dir-ltr"

NOTE: The static HTML documents may appear to be missing several elements or rearranged in an awkward or unexpected way. This is because the static documents do not contain extra CSS or JavaScript files that are necessary to make the site look how the designers wanted. If you'd like to see the page with HTML, CSS, and javascript elements see the Airbnb Search: Mission District, San Francisco, California search results page and an example Airbnb listing. You can then use the "Inspect element" feature of your browser to look at the HTML.

IMPORTANT NOTE ABOUT OPENING FILES:
If you are getting "encoding errors" while trying to open, read, or write from a file, add the following statement to any of your open() functions:

```
encoding="utf-8-sig"
```

An example of that within the function would be:

```
open("filename", "r", encoding="utf-8-sig")
```

There are a few special characters present on Airbnb's pages that aren't defined in standard UTF-8 (which is what Python runs by default). This is beyond the scope of what you have learned so far in this class, so we have provided this for you just in case this happens to you. Good luck!

Web Scraping & Accountability

Accountability is important for any democracy. Watchdogs and investigative journalists use web scraping from pages publicly available on the internet to hold corporations and governments socially accountable for their products and decisions.

For example, The Center for Investigative Reporting used web scraping to gather members of extremist Facebook hate groups and police Facebook groups. They then cross-referenced the two lists and confirmed hundreds of users that were both currently employed in law enforcement and a member of an online hate group. More than 50 police departments took action or launched internal investigations after the journalists called them with their findings.

Airbnb Terms of Service and Web Scraping

Airbnb's Terms of Service prohibit the use of automated data collection tools: *"Do not use bots, crawlers, scrapers, or other automated means to access or collect data or other content from or otherwise interact with the Airbnb Platform."*

However, the enforceability of this provision remains unsettled, and several high-profile U.S. court cases have debated the legality of web scraping. In this project, the webpages were accessed and downloaded manually rather than through automated tools, and therefore this process does not violate the site's Terms of Service.